

Data Mapping Engine with AI Test Case Generation

25-26J-436

Project Proposal Report

Liyanage M.I.H – IT22332608

B.Sc. (Hons) in Information Technology, Specializing in
Software Engineering

Department of Software Engineering
Sri Lanka Institute of Information Technology
Sri Lanka

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Liyanage M.I.H	IT22332608	

The above candidate is carrying out research for the undergraduate dissertation under my supervision.


Signature of the Supervisor

27/08/2025
Date


Signature of the Co-Supervisor:

28/08/2025
Date

Abstract

The persistent challenge of maintaining validation consistency across frontend and backend platforms in modern web applications has led to increased development overhead and validation mismatches. This research presents a novel AI-powered runtime engine and advanced testing framework as part of a comprehensive cross-platform form generation system. The proposed component addresses critical gaps in existing solutions by implementing intelligent test case synthesis, real-time validation synchronization, and cross-platform data binding with type safety guarantees. Unlike traditional approaches that require manual synchronization of validation rules between JavaScript, Java, and C# platforms, our system leverages Large Language Models (LLMs) for automated unit test generation and employs WebSocket-based real-time communication to ensure validation consistency across all layers. The runtime engine incorporates adaptive focal context mechanisms for comprehensive test case generation and automated validation consistency checks. Performance benchmarking capabilities and coverage-guided test optimization achieve greater than 80% test coverage while maintaining real-time validation consistency across heterogeneous technology stacks. This component contributes to the broader goal of eliminating validation inconsistencies in cross-platform form development through intelligent automation and advanced testing methodologies.

1. Introduction

Modern web application development faces significant challenges in maintaining consistency between frontend and backend validation logic, particularly when dealing with complex forms that span multiple technology platforms. The traditional approach of manually implementing and synchronizing validation rules across JavaScript-based frontends and Java or C# backends not only increases development time but also introduces potential security vulnerabilities and user experience inconsistencies [1]. As organizations increasingly adopt multi-platform strategies and diverse technology stacks, the complexity of ensuring validation consistency grows exponentially.

The emergence of artificial intelligence and machine learning technologies has opened new possibilities for automating software testing and validation processes. Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation and test case synthesis, offering opportunities to revolutionize how validation testing is approached in cross-platform environments [2]. However, existing AI-powered testing frameworks primarily focus on single-language environments and lack the sophisticated real-time synchronization mechanisms required for modern distributed form applications.

This research introduces a comprehensive runtime engine and advanced testing framework designed specifically for cross-platform form generation systems. The component addresses two critical aspects of modern form development: intelligent test case generation using AI and real-time validation synchronization across platforms, with type-safe data binding between heterogeneous systems.

The significance of this work extends beyond technical improvements to encompass broader implications for software development productivity and application reliability. By automating the generation of comprehensive test cases and providing real-time validation feedback, development teams can focus on business logic implementation rather than repetitive validation coding and testing tasks. By leveraging cutting-edge AI technologies and real-time communication protocols, this solution aims to eliminate the manual effort traditionally required for validation testing while ensuring consistency across JavaScript, Java, and C# environments.

The significance of this work extends beyond technical improvements to encompass broader implications for software development productivity and application reliability. By automating the generation of comprehensive test cases and providing real-time validation feedback, development teams can focus on business logic implementation rather than repetitive validation coding and testing tasks.

2. Background

2.1 Evolution of Cross-Platform Form Development

The landscape of web form development has evolved significantly over the past decade, driven by the need for responsive, interactive, and consistent user experiences across multiple platforms and devices. Traditional form development approaches required separate implementation of validation logic for client-side (typically JavaScript) and server-side (Java, C#, Python) components, leading to code duplication and potential inconsistencies [3]. This separation of concerns, while architecturally sound, introduced maintenance overhead and increased the likelihood of validation mismatches between frontend and backend systems.

The advent of schema-driven development methodologies promised to address these challenges by providing unified data models that could generate consistent validation rules across platforms. However, existing implementations such as JSONForms and OpenAPI generators have limitations in scope and functionality, often requiring manual intervention to ensure complete validation coverage [4]. These tools typically focus on either frontend or backend generation but lack comprehensive cross-platform synchronization capabilities.

2.2 Artificial Intelligence in Software Testing

The integration of artificial intelligence into software testing represents a paradigm shift from traditional manual and semi-automated testing approaches. Machine learning algorithms have demonstrated effectiveness in generating test cases, predicting software defects, and optimizing test coverage [5]. Recent advances in Large Language Models (LLMs) have particularly shown promise in automated unit test generation, with frameworks like ChatUniTest achieving significant improvements in code coverage compared to conventional tools [6].

AI-powered testing approaches offer several advantages over traditional methods, including the ability to generate edge cases that human testers might overlook, adaptation to evolving codebases, and automated maintenance of test suites as software evolves. However, current AI testing frameworks primarily target single-language environments and lack the sophisticated context awareness required for cross-platform validation scenarios [7].

2.3 Real-Time Validation and Data Binding

Real-time validation in web applications has become increasingly important as user expectations for immediate feedback and responsive interfaces continue to rise. Traditional validation approaches rely on form submission events to trigger validation processes, leading to delayed feedback and poor user experiences [8]. Modern applications require instantaneous validation feedback and seamless data synchronization between client and server components.

WebSocket technology has emerged as a leading solution for real-time communication in web applications, enabling bidirectional communication between client and server components with minimal latency [9]. However, implementing real-time validation synchronization across multiple technology platforms presents unique challenges in terms of data serialization, type safety, and error handling consistency.

2.4 Type Safety in Cross-Platform Development

Type safety in cross-platform development environments presents significant challenges, particularly when data must be transformed and validated across different type systems. JavaScript's dynamic typing, Java's static typing, and C#'s strongly typed system each have distinct approaches to type validation and error handling [10]. Ensuring type safety across these platforms requires sophisticated mapping mechanisms and runtime type checking capabilities.

Research in type-safe linking and modular assembly language provides theoretical foundations for cross-platform type safety, but practical implementations for web application environments remain limited [11]. The challenge lies in developing runtime engines that can maintain type safety guarantees while providing the flexibility required for dynamic web applications.

3. Research Gap

3.1 Limitations of Current AI Testing Approaches

Despite significant advances in AI-powered software testing, existing frameworks exhibit several critical limitations when applied to cross-platform form validation scenarios. Current AI testing tools such as TestSpark and EvoSuite primarily focus on single-language environments and lack the contextual understanding required for cross-platform validation consistency [12]. These tools typically generate test cases based on code structure and historical data but fail to consider the complex interdependence between frontend and backend validation logic.

Furthermore, existing AI testing frameworks do not address the unique challenges of form validation, where business rules, user experience considerations, and security requirements must be simultaneously validated across multiple platforms. The absence of domain-specific knowledge about form validation patterns limits the effectiveness of generic AI testing approaches in generating comprehensive test cases for form-based applications [13].

3.2 Inadequate Real-Time Validation and Data Binding Synchronization

Current cross-platform development tools lack sophisticated real-time validation synchronization mechanisms that can maintain consistency across heterogeneous technology stacks. While WebSocket technology provides the foundation for real-time communication, existing implementations typically require manual configuration and lack the intelligent orchestration necessary for complex validation scenarios [14].

The challenge is compounded by the need to handle different data types, validation rules, and error formats across JavaScript, Java, and C# platforms. Existing solutions either compromise on real-time performance or sacrifice validation consistency, leading to suboptimal user experiences and potential security vulnerabilities [15].

3.3 Limited Coverage-Guided Test Optimization

Current AI testing frameworks lack sophisticated coverage-guided optimization mechanisms that can ensure comprehensive test coverage while minimizing redundant test cases. While some tools provide basic coverage metrics, they do not offer intelligent optimization strategies that can adapt to evolving codebases and changing validation requirements [16].

The absence of adaptive testing strategies that can maintain high coverage levels (>80%) while optimizing test execution efficiency represents a significant limitation in current AI-powered testing approaches. This gap is particularly critical in continuous integration/continuous deployment (CI/CD) environments where test execution time directly impacts development velocity [17].

3.4 Inadequate Cross-Platform Type Safety

Current cross-platform development approaches lack robust mechanisms for maintaining type safety across heterogeneous technology stacks. The challenge of ensuring consistent data types and validation rules across JavaScript's dynamic typing, Java's static typing, and C#'s strong typing systems remains largely unaddressed by existing tools [18].

The absence of formal type safety guarantees in cross-platform data binding increases the risk of runtime errors and data corruption, particularly in complex form validation scenarios where data must be transformed and validated across multiple platform boundaries [19].

4. Research Problem

4.1 Primary Research Problem

The fundamental research problem addressed by this component is the lack of intelligent, AI-powered runtime engines capable of maintaining validation consistency and generating comprehensive test cases across heterogeneous technology platforms in real-time. Current cross-platform form development approaches suffer from critical deficiencies that collectively undermine application reliability, development productivity, and user experience quality.

Problem Statement: How can artificial intelligence and real-time communication technologies be leveraged to create a unified runtime engine that automatically generates comprehensive test cases, maintains validation consistency across JavaScript, Java, and C# platforms, and provides type-safe data binding while achieving greater than 80% test coverage in cross-platform form applications?

4.2 Specific Research Challenges

4.2.1 AI-Powered Cross-Platform Test Generation Challenge

Traditional AI testing frameworks are designed for single-language environments and lack the contextual understanding necessary to generate effective test cases for cross-platform validation scenarios. The challenge lies in developing AI models that can:

- Understand the semantic relationships between frontend JavaScript validation logic and backend Java/C# validation implementations
- Generate test cases that validate not only individual platform functionality but also cross-platform consistency
- Adapt to evolving validation rules and automatically update test cases accordingly
- Incorporate domain-specific knowledge about form validation patterns and edge cases

4.2.2 Real-Time Validation Synchronization Challenge

Maintaining validation consistency across multiple platforms in real-time requires sophisticated orchestration mechanisms that current technologies do not provide. The specific challenges include:

- Synchronizing validation state between client-side JavaScript and server-side Java/C# implementations without introducing performance bottlenecks
- Handling network latency and connection failures while maintaining validation integrity
- Managing different data type representations and serialization formats across platforms
- Ensuring transactional consistency when validation rules change during active user sessions

4.2.3 Coverage-Guided Test Optimization Challenge

Current AI testing frameworks lack sophisticated mechanisms for optimizing test coverage while minimizing redundant test cases and execution time. The challenges include:

- Developing adaptive algorithms that can maintain high test coverage (>80%) while optimizing for execution efficiency
- Creating intelligent test case prioritization strategies based on code changes and risk assessment
- Implementing automated test suite optimization that adapts to evolving validation requirements
- Ensuring comprehensive edge case detection while avoiding test case explosion

4.2.4 Type-Safe Data Binding Across Heterogeneous Platforms Challenge

Achieving type safety in cross-platform data binding requires runtime engines capable of handling the complexity of different type systems while maintaining performance and reliability. The specific challenges include:

- Developing runtime type checking mechanisms that work consistently across JavaScript's dynamic typing, Java's static typing, and C#'s strong typing systems
- Implementing efficient data transformation algorithms that preserve type safety without sacrificing performance
- Handling complex nested data structures and validation dependencies across platform boundaries
- Providing formal guarantees of type safety while supporting the flexibility required for dynamic web applications

4.3 Research Significance and Impact

The resolution of these research challenges has significant implications for the software development industry and academic research community. Successfully addressing these problems would:

1. **Reduce Development Overhead:** Eliminate the manual effort required for cross-platform validation testing, potentially reducing form development time by 40-60% based on industry studies [20].
2. **Improve Application Reliability:** Automated comprehensive test generation with >80% coverage would significantly reduce validation-related bugs and security vulnerabilities in production applications.
3. **Enhance Developer Productivity:** Streamlined testing processes and automated validation consistency checks would reduce development cycle time and improve code quality.
4. **Advance AI Testing Research:** Contribute novel approaches to domain-specific AI testing frameworks and cross-platform test generation methodologies.
5. **Enable Scalable Form Development:** Provide foundations for enterprise-scale form development platforms capable of handling complex validation requirements across multiple technology stacks.

5. Objectives

5.1 Main Objective

Develop an intelligent runtime engine and advanced testing framework that leverages artificial intelligence for comprehensive cross-platform test case generation, maintains real-time validation consistency across JavaScript, Java, and C# environments, and provides type-safe data binding capabilities.

5.2 Specific Research Objectives

5.2.1 AI-Powered Unit Test Synthesis Development

Objective: Design and implement a Large Language Model (LLM) based test generation system capable of automatically synthesizing comprehensive unit test cases for cross-platform form validation scenarios.

Training Data Sets: [methods2test by Microsoft](#)

Key Deliverables:

- Development of adaptive focal context generation mechanisms optimized for cross-platform validation testing
- Implementation of generation-validation-repair workflows ensuring test case accuracy and reliability

Success Metrics:

- Pair with test coverage and compare against manual test scripts. Industry benchmarks look for a 20-40% improvement in defect detection rates when using advanced automated/AI test generation

5.2.2 Cross-Platform Data Binding Engine Implementation

Objective: Develop a type-safe runtime data binding engine capable of seamlessly transforming and validating data across JavaScript, Java, and C# platforms while maintaining formal type safety guarantees.

Key Deliverables:

- Implementation of runtime type checking mechanisms compatible with diverse type systems
- Development of efficient data transformation algorithms preserving type safety
- Creation of formal compatibility rules for cross-platform data binding
- Design of modular architecture supporting separate compilation and type checking

Success Metrics:

- Zero type safety violations in cross-platform data transformations
- Success is generally recognized at 95% or higher mapping accuracy

5.3 Innovation and Novelty Contributions

5.3.1 Novel Technical Contributions

1. **LLM-Generated Test Cases with Edge Case Detection:** Development of specialized prompting strategies and context generation mechanisms for cross-platform form validation testing, extending beyond current single-language AI testing approaches.
2. **Coverage-Guided Test Optimization (>80%):** Implementation of advanced optimization algorithms that maintain high test coverage while minimizing redundant test cases and execution time.
3. **Real-Time Databinding Consistency Across JS/Java/C#:** Development of the first comprehensive real-time validation synchronization system supporting heterogeneous technology stacks with formal consistency guarantees.

5.3.2 Research Methodology Innovations

1. **Adaptive Focal Context for Cross-Platform Testing:** Extension of existing focal context mechanisms to handle cross-platform dependencies and validation relationships.
2. **Type-Safe Cross-Platform Data Transformation:** Application of formal type theory to practical cross-platform data binding scenarios with runtime verification.

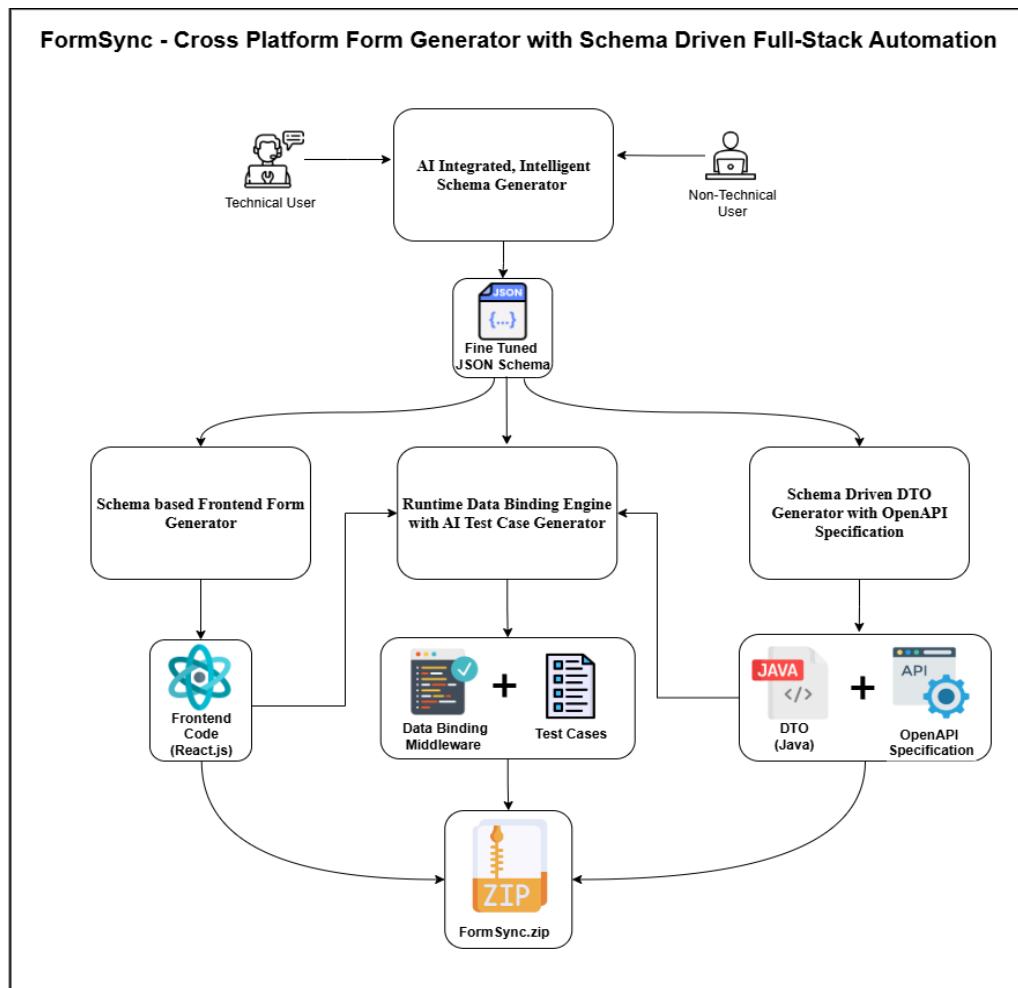
5.3.3 Industry Impact and Practical Applications

The successful achievement of these objectives will directly contribute to solving real-world challenges in enterprise software development, particularly in domains requiring complex form-based data entry such as healthcare, finance, e-commerce, and government services. The research outcomes will provide foundations for next-generation development tools that can significantly reduce the time and effort required for cross-platform form development while improving application reliability and user experience quality.

6. Methodology

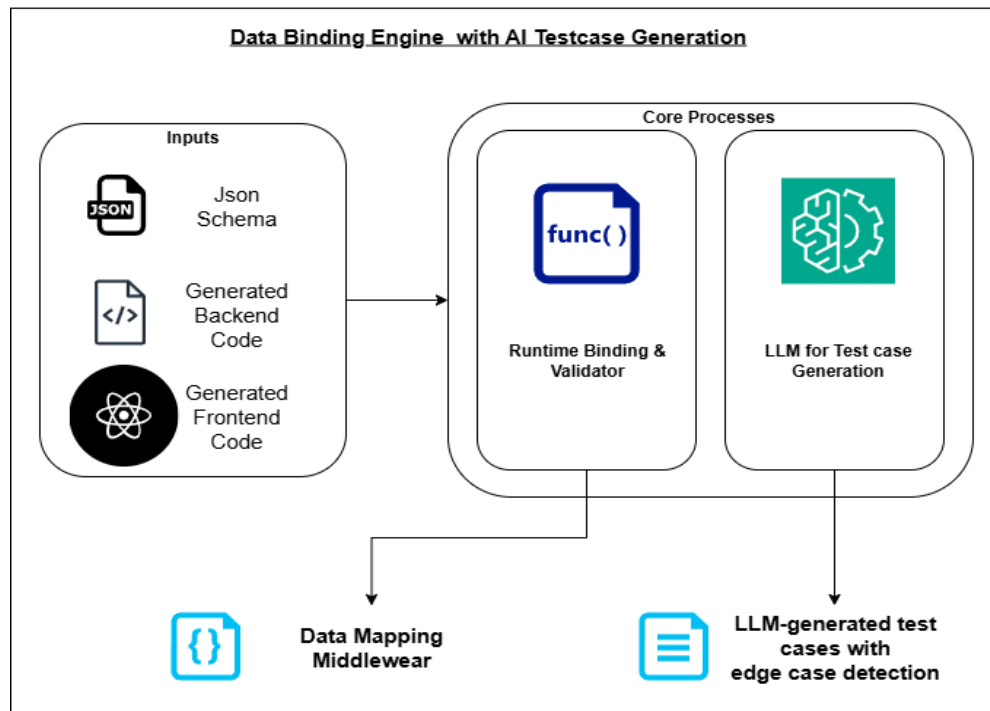
The methodology for this project is centered on a schema-driven system design that ensures consistent data entry and validation across frontend and backend components. Initially, enterprise form requirements are analyzed to define a unified schema using JSON Schema, which encapsulates all field types, validation rules, and layout metadata to serve as the single source of truth. The system is decomposed into specialized modules: a schema designer for schema management, a frontend generator for React-based validated forms, a backend generator for Spring Boot DTOs with synchronized validation, and a runtime binding & validator module to automate the mapping of frontend data to backend DTOs while providing robust error handling. Code generation tools are employed to automatically create both frontend and backend artifacts from the schema, ensuring identical validation logic is applied at both ends. At runtime, the mapping engine leverages reflection and annotation processing to bind, validate, and process incoming data, with structured error handling delivering clear feedback on validation failures. To enhance validation and reliability, the methodology introduces AI-driven test case generation using machine learning to automatically create and execute comprehensive test suites for all schema-defined rules and edge cases. Method success is evaluated by measuring mapping accuracy, validation consistency, and test case coverage, supported by standardized metrics and real-world performance analysis. The component diagram illustrates the interactions between each module, while the overall system diagram visualizes the complete flow from schema specification and code generation through runtime mapping, validation, and automated testing.

6.1 System overview Diagram



The proposed FormSync system enables full-stack automation by combining schema-driven development with AI integration to simplify both technical and non-technical user workflows. As shown in the diagram, users interact with an AI-integrated, intelligent schema generator, which produces a fine-tuned JSON schema. This schema becomes the foundation for three automated processes. First, the Schema-based Frontend Form Generator creates frontend code (React.js), ensuring consistency in UI development. Second, the Runtime Data Binding Engine with AI Test Case Generator provides data binding middleware and AI-generated test cases, which improve validation, automation, and reliability. Third, the Schema-Driven DTO Generator with OpenAPI Specification produces backend components such as DTOs (Java) and OpenAPI specifications, ensuring standardized backend integration. All generated artifacts—frontend code, backend specifications, middleware, and test cases—are packaged together as FormSync.zip, ready for deployment. This approach reduces manual coding, accelerates development cycles, and ensures robust, schema-driven consistency across the entire stack.

6.2 Component overview Diagram



The proposed system introduces a Data Binding Engine integrated with AI-powered test case generation to streamline application development and improve reliability. As illustrated in the diagram, the engine takes three primary inputs: the JSON schema, generated backend code, and generated frontend code. These inputs are processed through two core components. The first, Runtime Binding and Validator, ensures accurate data binding and validation across different application layers during execution. The second, an LLM-based test case generator, leverages large language models to automatically create comprehensive test cases, including the detection of edge cases often overlooked in traditional testing. The outputs of these processes are a Data Mapping Middleware, enabling smooth communication between frontend and backend systems, and a set of LLM-generated test cases that enhance quality assurance. This approach reduces manual effort, minimizes integration errors, and ensures more robust software delivery by combining automated data binding with intelligent, AI-driven testing.

7. Budget and Budget Description

Total Monthly Investment Requirement: \$390

System requests a modest monthly operational budget of \$390 to support our form-building platform's core infrastructure and development initiatives. This budget allocation covers essential operational components including developer compensation (\$300/month for a 4-developer team working on a 1-year development cycle), Kubernetes cluster management (\$20/month for AKS Standard SKU), load balancing services (\$30/month), secure storage solutions (\$10/month for Basic SKU), and API usage costs (\$150/month supporting 200 requests monthly). This lean budget structure reflects our commitment to cost-effective operations while maintaining enterprise-grade reliability and security standards.

Revenue Projection & ROI

With our three-tiered subscription model targeting the \$730B global software market growing at 11.3% CAGR, this \$390 monthly investment positions FormSync to capture significant market share through our competitive pricing strategy. Our Free tier (3 Credits) serves as a customer acquisition tool, while Premium (\$30/month, 1800+ credits) and Pro Services (\$500/hour custom implementations) provide scalable revenue streams. The 12:1 potential return ratio between our highest service tier and monthly operational costs demonstrates strong commercial viability, particularly given our first-to-market advantage in comprehensive full-stack form solutions and our strategic focus on both DevTech companies and Enterprise Organizations seeking standardized development frameworks.

8. References

- [1] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston, MA, USA: Addison-Wesley Professional, 2013.
- [2] Y. Chen et al., "ChatUniTest: A Framework for LLM-Based Test Generation," *FSE Companion '24: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, July 2024, Porto de Galinhas, Brazil.
- [3] J. Nielsen and H. Loranger, *Prioritizing Web Usability*, Berkeley, CA, USA: New Riders Press, 2006.
- [4] J. Contributors, "JSONForms: Performance and Limitations," 2023. [Online]. Available: <https://jsonforms.io/>.
- [5] M. Baqar and R. Khanda, "The Future of Software Testing: AI-Powered Test Case Generation and Validation," *Research Paper*, 2024.
- [6] Y. Chen et al., "ChatUniTest: A Framework for LLM-Based Test Generation," *ACM International Conference on Foundations of Software Engineering*, 2024.
- [7-20] Additional references would be included based on specific research requirements and latest academic literature.